

Prompt Engineering vs Context Engineering vs Harness Engineering

Overview

Prompt engineering focuses on writing effective instructions for the language model. Context engineering focuses on selecting and organizing the right information that the model receives. Harness engineering focuses on building the infrastructure and workflow around the model, including tools, memory, retrieval, orchestration, logging, monitoring, and error handling.

Comparison

Prompt Engineering

Designing instructions for the LLM. Answers: 'How should I ask the model?'

Context Engineering

Selecting, ranking, and organizing the information sent to the LLM (retrieved documents, memory, chat history, system instructions). Answers: 'What should the model know?'

Harness Engineering

Building the complete AI application that coordinates tools, retrieval, workflows, retries, permissions, memory, and output formatting. Answers: 'How should the whole AI system operate?'

Research Agent Example

Prompt Engineering: 'Compare LangGraph and CrewAI. Use a table and mention strengths and weaknesses.'

Context Engineering: Retrieve only the most relevant LangGraph and CrewAI documentation, include relevant chat history, and exclude unrelated papers.

Harness Engineering: Accept the query, choose tools, retrieve documents, assemble context, call the LLM, format the response, save history, and generate reports.

Visual Flow

User Query → Harness Engineering → Retrieve Documents/Tools → Context Engineering → Prompt Engineering → LLM → Response

Key Takeaways

- Prompt engineering is about **how you ask**.
- Context engineering is about **what the model knows**.
- Harness engineering is about **how the entire AI system works**.

Modern AI applications rely on all three, with context and harness engineering becoming increasingly important for production-grade agentic systems.